
Path Planning Task 2

Hla Ahmed Mahmoud Mohamed

Assumptions

I assumed that the minimum path width would be 3 m for the virtual cone projection , I also assumed that the maximum distance the car can move without seeing(straight ahead distance) is 5 m which fits with the requirement of the path being at least 5 m and the max distance between two cones on one side is 5m . I also assumed that points behind the car aren't required

```
HALF_W=1.5 # i assumed the minimum track width to be 3 meters
Max_DIFFERENCE=5 # i assumed the maximum distance between two cones to be 5
meters
straight_ahead_distance=5
```

Algorithm Used

I used the centerline algorithm but I didn't follow it exactly like in the slides. The steps were the following: 1. sort the cones from nearest to farthest from car position, 2. separate into left and right cones with the color ,3. pair the cones (blue with yellow) that can be paired based on maximum assumed separation between cones on the same side ,4. Get the midpoints of the pairs ,5 if there is still points that haven't been matched use the virtual cone idea, 6 if there is no points to get information from use the cars heading ,7. Sort again because the points aren't added in order, 8. Generate the points between the car and midpoints with step =0.5 , 9.lastly extend the path if it didn't reach 5m

Helper functions to make code clearer

Get_midpoint : gets midpoint between two cones

Get_distance: gets how far ahead a point is from the car (cant tell the difference is the points are on the same horizontal line)

Get_virtual_center: places a point where the missing cone on the other side would be. It normalizes (vx, vy) (cone1.x-cone2.x,cone1.y-cone2.y) to a unit vector and rotates it 90°. A blue (left) cone is rotated to the right of the travel direction, and a yellow (right) cone is rotated to the left (perpendicular to the original vector), each by half the width length (midpoint between cone and virtual cone). The side is decided entirely by the sign of (vx, vy), so the order of points is very important

Sort_cones: removes points behind the car and sorts them based on distance ahead (get_distance)

Get_pairs: matches the blue and yellow cones with the smallest difference between them with each one being only paired once while being under the maximum allowed distance and returning the unmatched cones also

If_nocorcones: it sorts them from nearest to car using hypotenuses length then checks if only one point the direction of path is assumed from car heading and if multiple cones , the vector

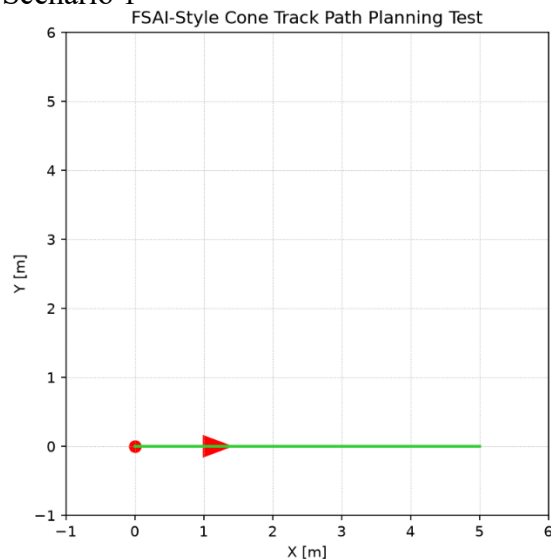
components are taken or vx and vy , it send them to get the midpoint using get_virtual_center(for both cases just different ways to get track direction)

Add_points: makes intermediate points between car pose and cones or cones with each other with maximum step size =0.5 for a complete path

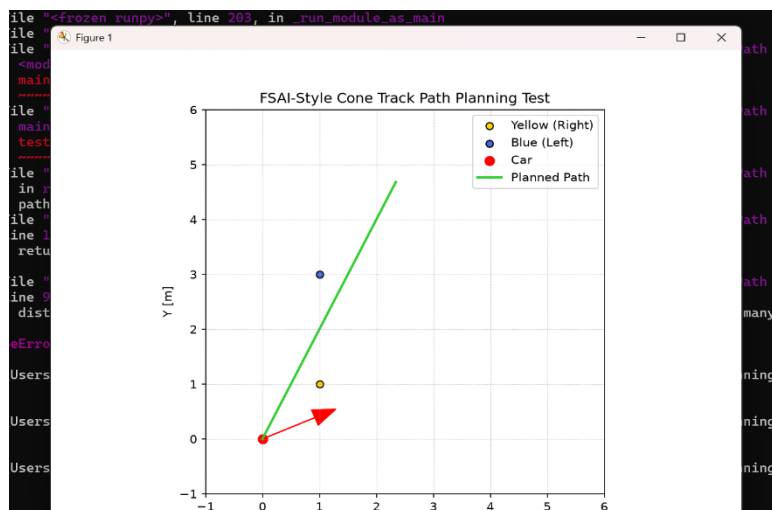
Continue_path: extends the path from last two cones direction to continue path until 5 m

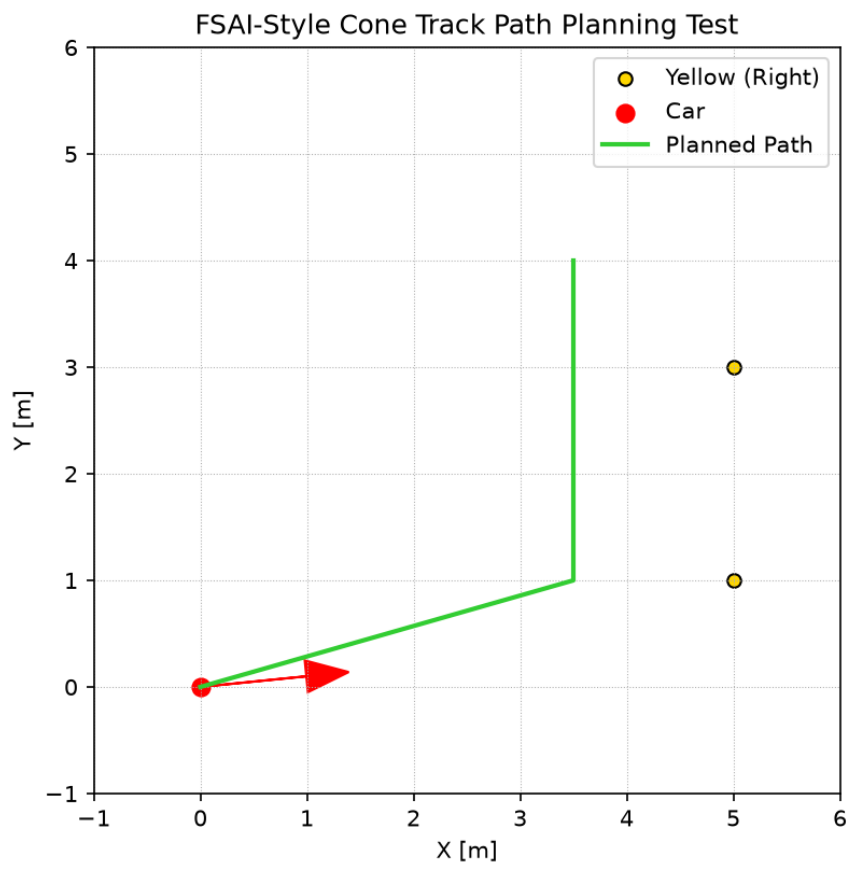
Where my implementation mainly differed was the pairing part as in the slides multiple cones could be paired to the same cone from the other side but in my case only one cone can match with another cone from the other side (the approach I used is simpler but the one in the slides is more accurate)

Scenario 1

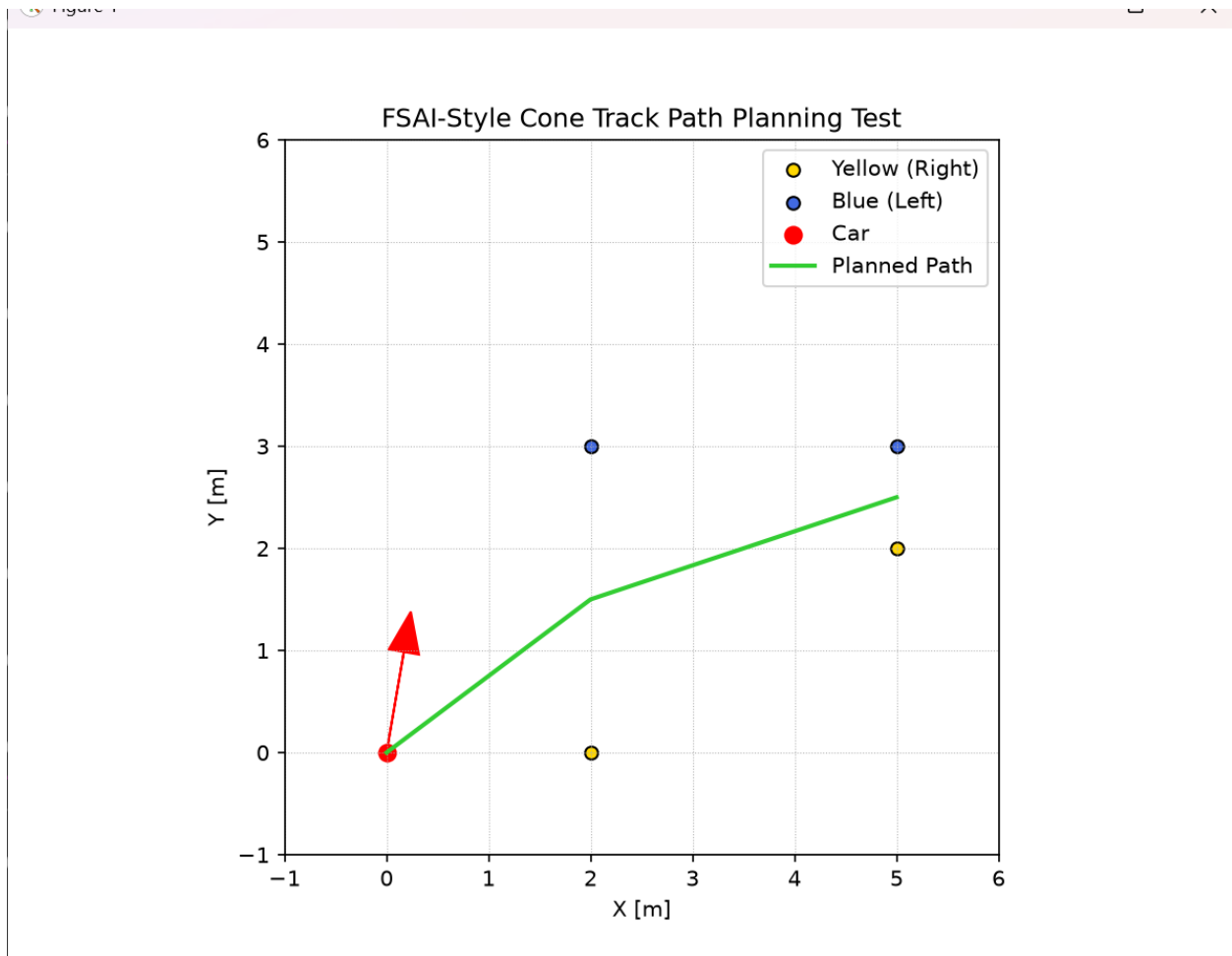


scenario 2



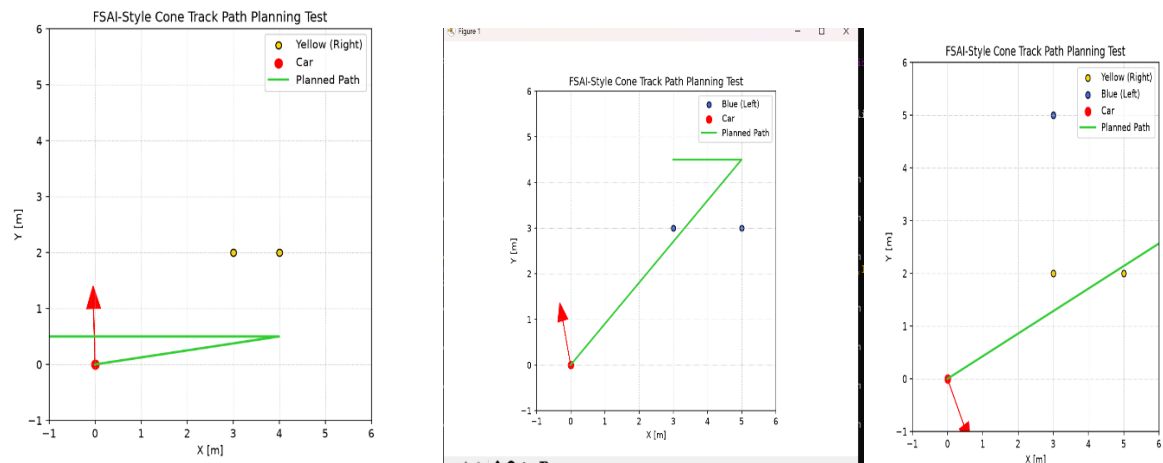


Scenario 13



Scenario 19

But where I mainly saw problems was with scenarios like 5 , 8,14 and a couple others

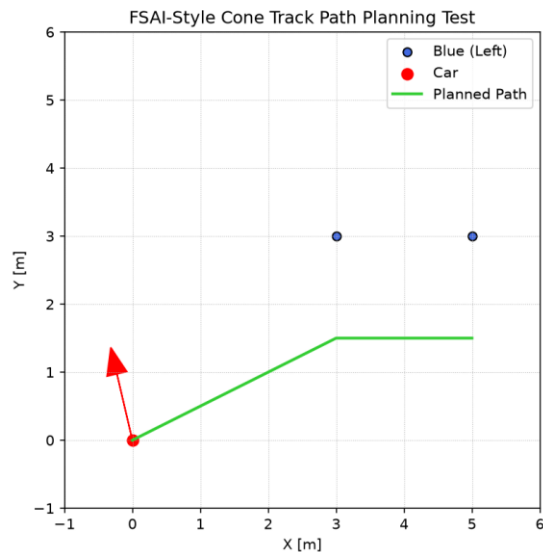
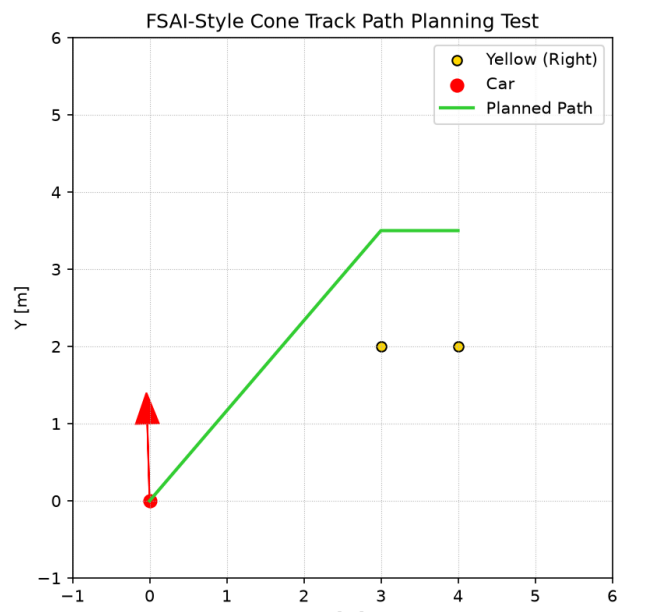


The reasons where the following:

1. When two cones were on the same side, the track direction is estimated as the vector from one cone to the next. This vectors sign depends on which cone is treated as "first". When the cones are side by side relative to the car, their order isn't always accurate because it was calculated based on heading, so the sign can be backward. The virtual cones are made by rotating this vector by 90° , so a flipped sign puts them on the opposite side of the cones.
2. The final midpoints were sorted by distance along the car's heading. For a track that has mostly sideways points, the points have nearly the same value, so the order is inaccurate and the path jumped back and forth

Therefore I made sure to sort the virtual cones and midpoint by the hypotenuses

After fixing
scenario 5 and 8 after fixing

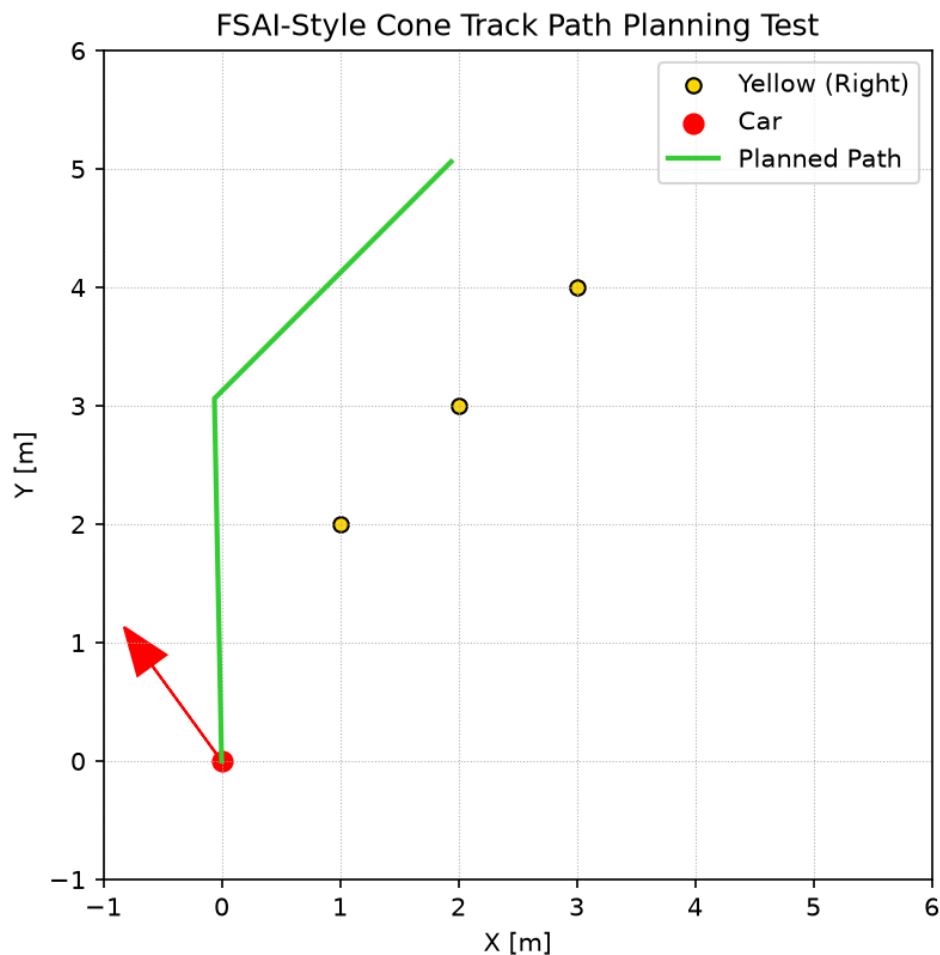


With 14 , the problem was that the cones where behind the car which I made the car ignore in the code so this a limitation I cant solve in the meantime because thing this would affect the rest of the logic so I will assume any path that's supposed to be behind the car isn't something we care for

Part 2

Nothing extra was added for the 3 cones on side problem as the alogrithim already handles missing cones with virtual ones but a new scenario was added

```
"21": (  
    [Cone(x=1.0, y=2.0, color=0), Cone(x=2.0, y=3.0, color=0), (Cone(x=3.0,  
y=4.0, color=0))],  
    CarPose(x=0.0, y=0.0, yaw=2.2),  
),
```



Limitations

1. Ordering by distance can cause problems in tight curves
2. Cones behind the car are ignored (but realistic for real life track)
3. Wrong color will change the computed path drastically (aka fail)
4. Needs smoothing very rigid
5. Not the most accurate because of pairing decision is 1:1